



MANUAL DE USUARIO

GoLite Compilador – Fase 1

Organización de Lenguajes y Compiladores 1

Universidad de San Carlos de Guatemala

Yerri Jonatan Gadiel Bamaca Lopez

202132066

Manual de Usuario – GoLite IDE

GoLite IDE es un entorno que Incluye editor de código, consola de salida y generador de reportes automáticos.

Requisitos del sistema

Java JDK 17 o superior instalado

Sistema operativo: Windows, Linux o macOS

RAM mínima: 512 MB

Pantalla: resolución mínima 1024 × 600

Abrir con: `java -jar` (y el link del archivo. Jar que lanza)

Iniciar la aplicación

Compilar el proyecto en NetBeans con **F11**

Ejecutar con **F6**, o bien directamente:

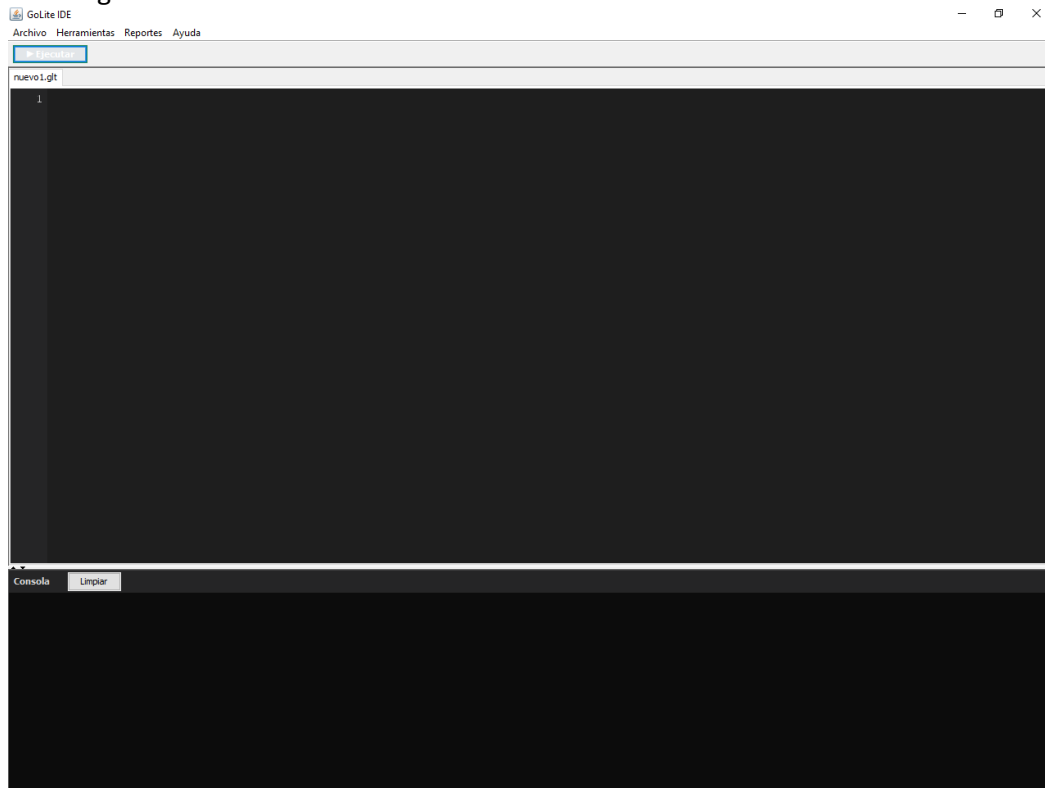
```
java -jar GoLiteCompiler.jar
```

Al iniciar aparece la ventana principal con una pestaña vacía lista para escribir.

Interfaz principal

+toolbar		Archivo Herramientas Reportes Ayuda Ejecutar (F5)	Barra de menú
nuevo1.glt x main.glt x			Pestañas de
código	1	func main(){	Editor de
	2	fmt.Println("Hola GoLite")	
	3	}	
salida	Consola [Limpiar]		Consola de
	> Ejecutando main.glt ...		
	Hola GoLite		
	[RESUMEN] Compilación e interpretación exitosa.		
Ejecución exitosa – main.glt			Barra de estado

Vista grafica:



Gestión de archivos

Crear un archivo nuevo

Menú **Archivo** → **Nuevo** o **Ctrl+N**

Se abre una nueva pestaña vacía con nombre nuevoN.glt

Abrir un archivo existente

Menú **Archivo** → **Abrir...** o **Ctrl+O**

Selecciona un archivo .glt del disco

El archivo se abre en una nueva pestaña

Guardar

Ctrl+S guarda el archivo activo

Si es nuevo (sin nombre de disco), abre el diálogo "Guardar como"

Los archivos con cambios sin guardar muestran * en el título:
main.glt *

Guardar como

Menú **Archivo** → **Guardar como...**

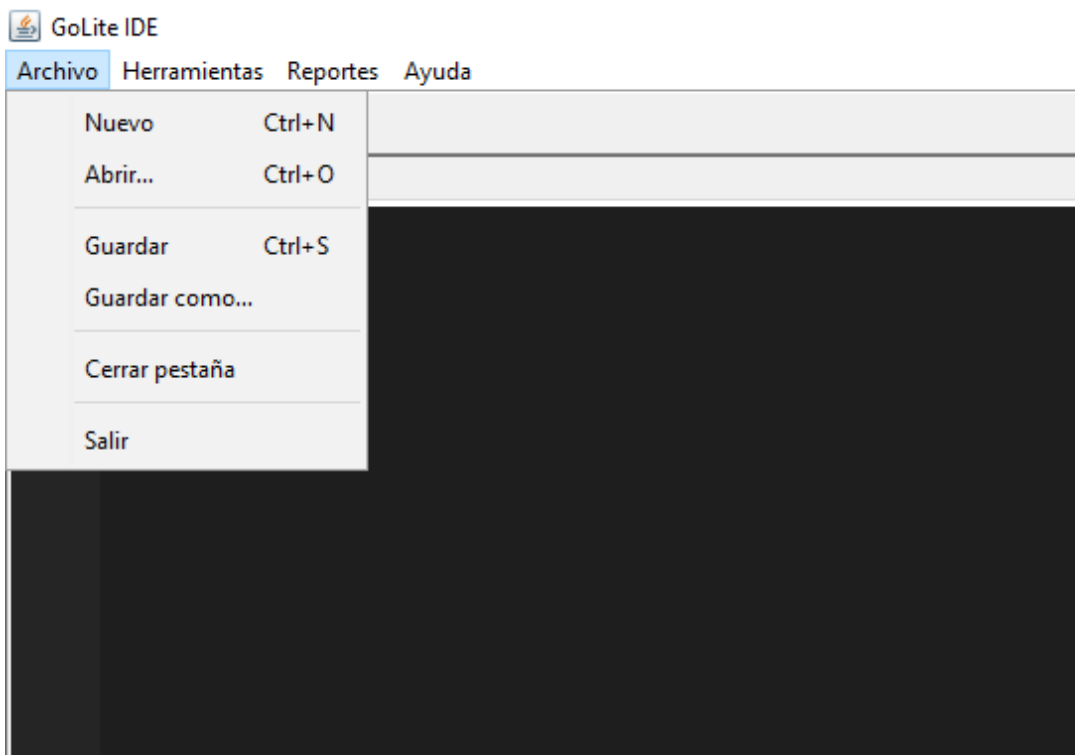
La extensión .glt se agrega automáticamente si no la escribes

Cerrar una pestaña

Click en el botón × de la pestaña

O menú **Archivo** → **Cerrar pestaña** (**Ctrl+W**)

Si hay cambios sin guardar, el IDE pregunta si deseas guardar antes



Escribir código GoLite

El editor soporta:

Números de línea en la columna izquierda (la línea activa aparece más brillante)

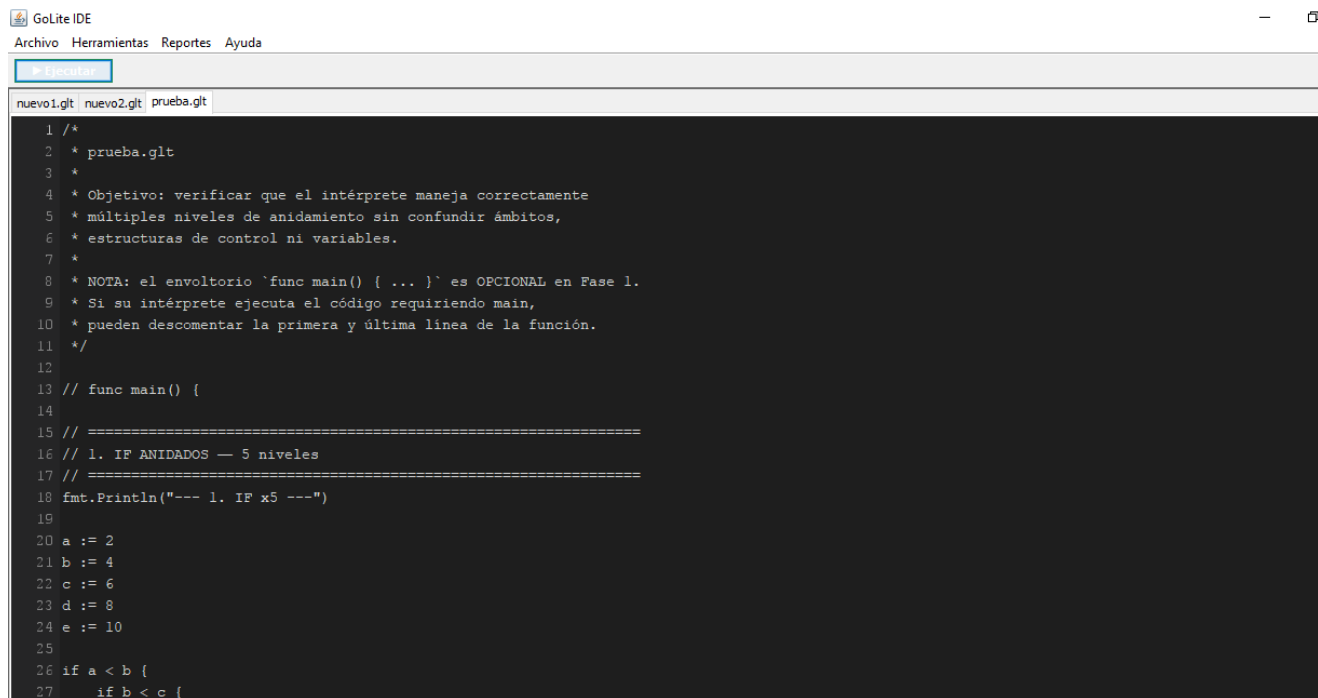
Scroll horizontal y vertical

Selección de texto con mouse o teclado

Copiar / Pegar con Ctrl+C / Ctrl+V

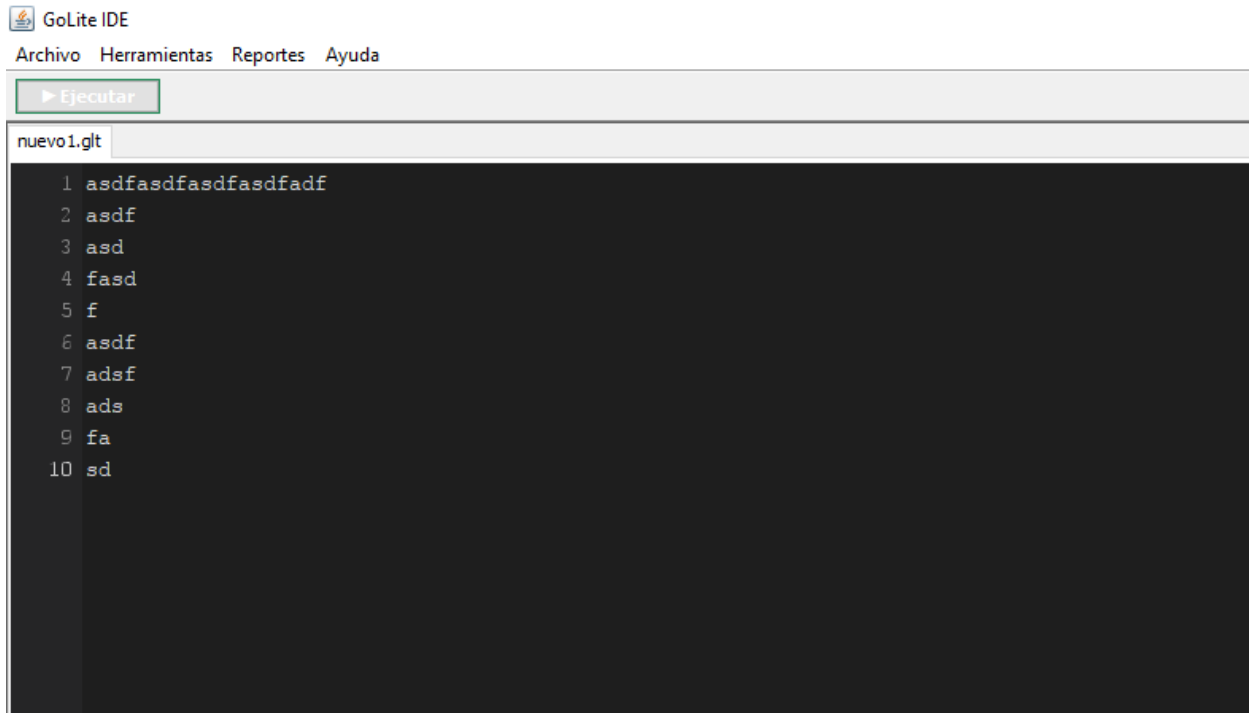
Deshacer / Rehacer con Ctrl+Z / Ctrl+Y

Tabs de 4 espacios



The screenshot shows the GoLite IDE interface. At the top, there is a title bar with the GoLite logo and the text "GoLite IDE". Below the title bar is a menu bar with the following items: "Archivo", "Herramientas", "Reportes", and "Ayuda". Below the menu bar is a toolbar with a single button labeled "Ejecutar". Below the toolbar is a tab bar with three tabs: "nuevo1.glt", "nuevo2.glt", and "prueba.glt". The "prueba.glt" tab is selected. The main area is a code editor with a dark background. The code is written in Go and includes line numbers on the left side. The code is as follows:

```
1 /*
2  * prueba.glt
3  *
4  * Objetivo: verificar que el intérprete maneja correctamente
5  * múltiples niveles de anidamiento sin confundir ámbitos,
6  * estructuras de control ni variables.
7  *
8  * NOTA: el envoltorio `func main() { ... }` es OPCIONAL en Fase 1.
9  * Si su intérprete ejecuta el código requiriendo main,
10 * pueden descomentar la primera y última línea de la función.
11 */
12
13 // func main() {
14
15 // =====
16 // 1. IF ANIDADOS — 5 niveles
17 // =====
18 fmt.Println("--- 1. IF x5 ---")
19
20 a := 2
21 b := 4
22 c := 6
23 d := 8
24 e := 10
25
26 if a < b {
27     if b < c {
```



Ejemplo de programa mínimo

```
func main() {  
    fmt.Println("Hola GoLite")  
}
```

Ejecutar el programa

Escribe o abre tu archivo .glt

Presiona **F5** o el botón **Ejecutar**

La consola muestra la salida del programa

El proceso completo que ocurre al presionar Ejecutar:

Análisis léxico → Análisis sintáctico → Intérprete → Salida en consola

Si hay errores, la consola muestra mensajes [WARN] o [ERROR] y la barra de estado indica X Errores encontrados.

Reportes

Los reportes están disponibles en el menú **Reportes** después de ejecutar. Se abren en ventanas HTML con colores.

Reporte de Errores – Ctrl+Alt+E

Muestra todos los errores encontrados:

No.	Descripción	Línea	Columna	Tipo
1	El símbolo "-" no es reconocido	5	3	léxico
2	La variable 'x' no está declarada	12	8	semántico

Tabla de Tokens – Ctrl+Alt+T

Muestra cada token reconocido por el analizador léxico:

No.	Lexema	Tipo	Línea	Columna
1	func	RES_FUNC	1	1
2	main	IDENTIFICADOR	1	6

Tabla de Símbolos – Ctrl+Alt+S

Muestra todas las variables y funciones declaradas:

No.	ID	Tipo símbolo	Tipo dato	Ámbito	Línea	Col
1	main	funcion	void	Global	1	1
2	x	variable	int	main	3	5

Reporte AST – Ctrl+Alt+A

Muestra el Árbol de Sintaxis Abstracta generado por el parser:

```
Programa
  Funcion: main
    Bloque
      FmtPrintln
        Literal(string): Hola GoLite
```

Sintaxis del lenguaje GoLite

Variables


```

var edad    int    = 25
var nombre  string = "Ana"
var activo  bool    = true
var pi      float64 = 3.14159
var letra   rune    = 'A'

```

```

// Declaración corta (infiere el tipo)
ciudad := "Guatemala"

```

Operadores

```

// Aritméticos
10 + 3    // 13
10 - 3    // 7
10 * 3    // 30
10 / 3    // 3 (enteros: trunca)
10 % 3    // 1

```

```

// Asignación compuesta
x += 5    // x = x + 5
x -= 3    // x = x - 3
x++       // x = x + 1
x--       // x = x - 1

```

```

// Comparación
a == b    a != b    a < b    a > b    a <= b    a >= b

```

```

// Lógicos
a && b    a || b    !a

```

Condicionales

```

if x > 10 {
    fmt.Println("mayor")
} else if x == 10 {
    fmt.Println("igual")
} else {
    fmt.Println("menor")
}

```

Bucles

```

// For estilo while
for i <= 10 {
    fmt.Println(i)
    i++
}

// For clásico
for i := 0; i < 10; i++ {
    fmt.Println(i)
}

```

Break y Continue

```
for i := 0; i < 10; i++ {  
    if i == 3 { continue } // Salta esta iteración  
    if i == 7 { break }    // Sale del bucle  
    fmt.Println(i)  
}
```

Funciones

```
// Sin retorno  
func saludar(nombre string) {  
    fmt.Println("Hola,", nombre)  
}
```

```
// Con retorno  
func suma(a int, b int) int {  
    return a + b  
}
```

```
// Llamada  
saludar("GoLite")  
resultado := suma(3, 7)
```

Funciones embebidas

```
fmt.Println("texto", 42, true) // Imprime en consola  
num := strconv.Atoi("123")     // String → int  
flt := strconv.ParseFloat("3.14") // String → float64  
tipo := reflect.TypeOf(variable) // Tipo como string
```

Comentarios

```
// Comentario de una línea
```

```
/*  
    Comentario  
    multilínea  
*/
```

Mensajes de la consola

Mensaje	Significado
[INFO]	Información del proceso de compilación
[OK]	Fase completada sin errores
[WARN]	Advertencia: hay errores pero la ejecución continuó parcialmente

Mensaje	Significado
[ERROR]	Error que detuvo la ejecución
[RESUMEN]	Resultado final de la compilación

Errores comunes y soluciones

Error	Causa	Solución
La variable 'x' no está declarada	Usas una variable antes de declararla	Declara con <code>var x int 0</code> o <code>x := valor</code>
No se puede dividir entre cero	Divisor es 0	Verifica que el denominador no sea 0
Operación '+' no válida entre int y bool	Tipos incompatibles	Usa el mismo tipo en la operación
El símbolo '-' no es reconocido	Carácter inválido en el código	Elimina caracteres especiales
La función 'foo' no está declarada	Llamas a función inexistente	Declara la función antes de llamarla
'break' fuera de un bucle	Break fuera de for/switch	Usa break solo dentro de for
strconv.Atoi: no puede convertir "3.14"	Decimal en Atoi	Usa strconv.ParseFloat para decimales

The screenshot shows the GoLite IDE interface. The top menu bar includes 'Archivo', 'Herramientas', 'Reportes', and 'Ayuda'. Below the menu is a toolbar with a 'Ejecutar' button. The main editor window displays a Go file named 'prueba.glt' with the following code:

```

1 /*
2  * prueba.glt
3  *
4  * Objetivo: verificar que el intérprete maneja correctamente
5  * múltiples niveles de anidamiento sin confundir ámbitos,
6  * estructuras de control ni variables.
7  *
8  * NOTA: el envoltorio `func main() { ... }` es OPCIONAL en Fase 1.
9  * Si su intérprete ejecuta el código requiriendo main,
10 * pueden descomentar la primera y última línea de la función.
11 */
12
13 // func main() {
14
15 // =====
16 // 1. IF ANIDADOS — 5 niveles
17 // =====
18 fmt.Println("--- 1. IF x5 ---")
19
20 a := 2
21 b := 4
22 c := 6
23 d := 8
24 e := 10
25
26 if a < b {
27     if b < c {
28         if c < d {
29             if d < e {

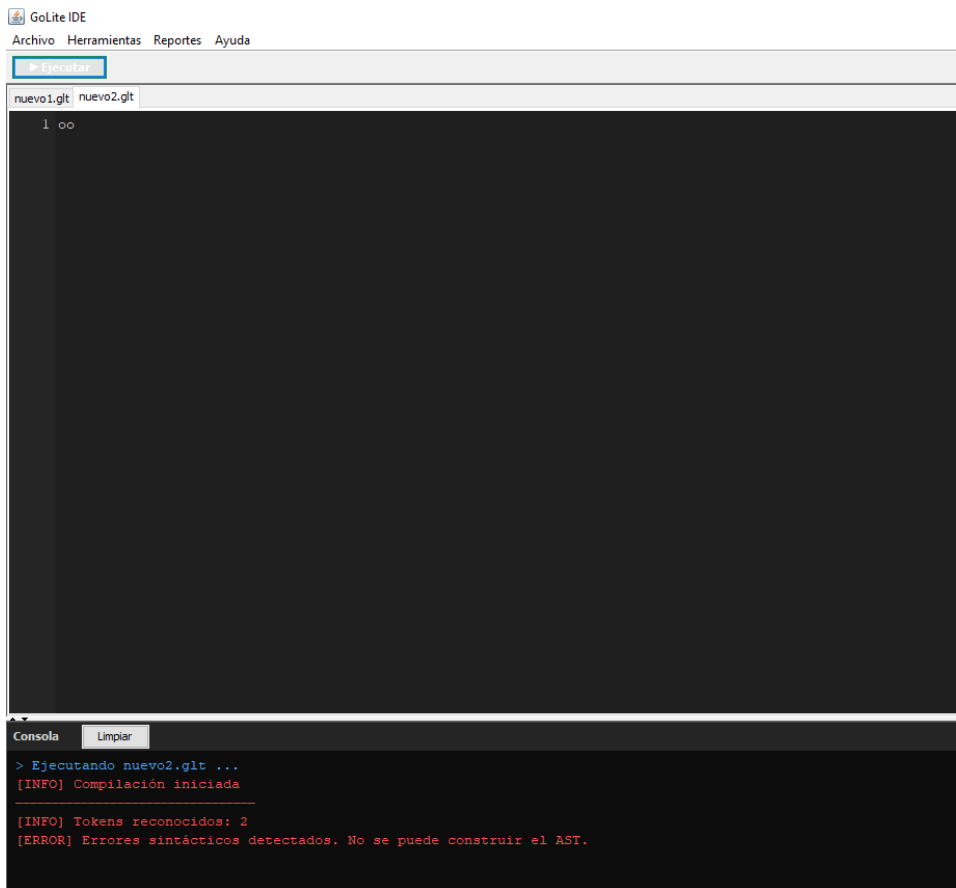
```

The console window at the bottom shows the execution output:

```

> Ejecutando nuevo2.glt ...
[INFO] Compilación iniciada
[INFO] Tokens reconocidos: 2
[ERROR] Errores sintácticos detectados. No se puede construir el AST.

```



Atajos de teclado

Acción	Atajo
Nuevo archivo	Ctrl+N
Abrir archivo	Ctrl+O
Guardar	Ctrl+S
Cerrar pestaña	Ctrl+W
Ejecutar	F5
Reporte de Errores	Ctrl+Alt+E

Tabla de Tokens	Ctrl+Alt+T
Tabla de Símbolos	Ctrl+Alt+S
Reporte AST	Ctrl+Alt+A



GoLite IDE

Archivo Herramientas Reportes Ayuda

▶ Ejecutar

nuevo1.glt

nuevo2.glt

1 |